FYEO

Aureus Ox

Repo: https://github.com/Aureus-Ox/aureus-ox-venmo-foundry

Security Review Update: March 20, 2026

Reviewer: reviewer@gofyeo.com

Security Code Review TimelockedVaultWrapper

New security issues, 7

After the development team implemented the latest updates, FYEO conducted a review of the modifications. The primary goal of this evaluation was to ensure the continued robustness of the program's security features, safeguarding user funds and maintaining the overall integrity of the program.

General Updates:

The TimelockedVaultWrapper is a smart contract that acts as a middleman between users and an underlying investment vault (Upshift vault). When users deposit funds, the wrapper forwards them into the underlying vault and gives users shares representing their stake. The key difference from interacting with the vault directly is that withdrawals are time-locked. When a user wants to get their money back, they first submit a redemption request and receive a future date on which they can actually collect their funds. This adds a waiting period to withdrawals, which is common in DeFi to give the vault time to unwind positions safely.

On top of the time-locked withdrawal flow, the wrapper also handles a management fee that accrues over time and goes to a designated fee recipient, an instant redemption option that lets users skip the waiting period in exchange for paying a commission fee, and an emergency mode that the contract owner can activate to let everyone pull their funds out immediately if something goes wrong. Essentially, it wraps a simpler vault with extra guardrails around timing, fees, and emergency access.

Claim does not zero out pending claims on the success path

Finding ID: FYEO-AUOX-01
Severity: **Critical**
Status: **Remediated**

Description

In the `try` branch of `claim()` (successful claim from the underlying vault), the `_pendingClaims` mapping for the user is never cleared. Only the `catch` branch zeroes it out. Since the `require(pendingClaims > 0)` check keeps passing, a user can call `claim()` repeatedly for the same epoch and drain the contract's asset balance.

Proof of Issue

File name: TimelockedVaultWrapper.sol
Line number: 224

```
try $ .upshiftVault.claim(year, month, day, address(this)) returns (uint256
sharesBurned, uint256 assetsClaimed) {
    // perform claim and transfer the underlying assets back to the user
    IERC20(asset()).safeTransfer(receiverAddr, assetsClaimed);
    assets = assetsClaimed;
}
```

Severity and Impact Summary

Allows repeated claiming against the same epoch. An attacker can drain the contract's entire asset balance by calling `claim()` in a loop.

Recommendation

Set `_pendingClaims[claimableEpoch][receiverAddr] = 0` before the `try` block or inside the success branch.

First user to claim an epoch receives ALL users assets for that epoch

Finding ID: FYEO-AUOX-02
Severity: **Critical**
Status: **Remediated**

Description

Because the wrapper is always the `receiverAddr` in the underlying vault, calling `vault.claim(year, month, day, address(this))` processes the entire burnable amount for the wrapper in one shot — burning all shares from every user who requested a redeem for that epoch and sending all corresponding assets to the wrapper. The wrapper then forwards the entire `assetsClaimed` to the single caller.

When the second user calls `claim()`, the vault reverts with `NoSharesForReceiver` (already processed), hitting the `catch` branch which attempts to send `_pendingClaims[epoch][user]` from the wrapper's balance, but those assets may already have been drained by the first claimer.

Proof of Issue

File name: TimelockedVaultWrapper.sol
Line number: 224

```
try $ .upshiftVault.claim(year, month, day, address(this)) returns (uint256
sharesBurned, uint256 assetsClaimed) {
    // perform claim and transfer the underlying assets back to the user
    IERC20(asset()).safeTransfer(receiverAddr, assetsClaimed);
    assets = assetsClaimed;
}
```

Severity and Impact Summary

The first user to call `claim()` for a given epoch receives all assets belonging to every user who requested a redeem for that epoch. Subsequent claimers lose their funds.

Recommendation

Do not forward the vault's `assetsClaimed` directly to a single user. Use the wrapper's own `_pendingClaims` accounting to determine each user's share, and treat the vault-level claim as a batch settlement.

Claim catch branch treats all reverts as "already claimed"

Finding ID: FYEO-AUOX-03
Severity: **High**
Status: **Remediated**

Description

When the underlying vault's `claim()` reverts, the `catch` block assumes the vault already processed the claim and the assets are sitting in the wrapper. However, the vault can revert for several distinct reasons: `NoSharesForReceiver`, `TooEarly`, `WithdrawalsPaused`, `NotConfigured`, and `ifSenderWhitelisted`.

If the vault reverts for any reason other than "already claimed", the wrapper will attempt to transfer `_pendingClaims` worth of assets from its own balance. This can misallocate funds belonging to other users, or if insufficient balance exists, the transfer reverts but `_pendingClaims` is still zeroed, destroying the user's claim record while the underlying shares were never redeemed.

Additionally, any address can call `claim()` with any `receiverAddr`, creating a griefing vector.

Proof of Issue

File name: TimelockedVaultWrapper.sol
Line number: 228

```
} catch {
    // if this fails the underlying vault contract has already processed the
claim
    // forward the assets back to the end user
    assets = $ ._pendingClaims[claimableEpoch][receiverAddr];
    IERC20(asset()).safeTransfer(receiverAddr, assets);

    $ ._pendingClaims[claimableEpoch][receiverAddr] = 0;
    return assets;
}
```

Severity and Impact Summary

Fund misallocation or permanent loss of claim records. A blanket catch on a multi-revert-reason external call leads to incorrect state transitions under non-happy-path conditions.

Recommendation

Do not use a blanket `catch`. Either check specific revert reasons, or track whether the underlying claim was already executed via a separate boolean flag.

Claim date comparison logic is broken

Finding ID: FYEO-AUOX-04
Severity: **High**
Status: **Remediated**

Description

The `claim()` function compares year, month, and day independently rather than as a composite date. This causes it to incorrectly reject valid claims where the claimable day-of-month is larger than the current day-of-month, even if the current date is months or years later. For example, a claim for 2025-12-15 attempted on 2026-01-10 fails because `blockMonth (1) >= month (12)` is false. Users can have funds permanently locked.

The underlying Upshift vault has its own `TooEarly` check using proper epoch comparison, so the wrapper's broken check is strictly more restrictive and blocks legitimate claims the vault would allow.

Proof of Issue

File name: TimelockedVaultWrapper.sol
Line number: 221

```
require(blockYear >= year && blockMonth >= month && blockDay >= day);
```

Severity and Impact Summary

Users' funds can become permanently locked. Any claim where the claimable day-of-month exceeds the current day-of-month will be rejected regardless of how far in the future the current date actually is.

Recommendation

Compare the raw epoch timestamps directly instead of decomposing into year/month/day components.

Inherited ERC4626 functions enable griefing

Finding ID: FYEO-AUOX-05
Severity: **Medium**
Status: **Remediated**

Description

`TimelockedVaultWrapper` overrides `deposit` and `totalAssets` but leaves the base ERC4626 `mint`, `withdraw`, and `redeem` publicly callable. The inherited `mint` transfers collateral directly into the wrapper (bypassing the Upshift vault) while minting shares. Since `totalAssets()` only counts Upshift LP value, this inflates `totalSupply` without increasing `totalAssets()`, diluting all existing depositors.

This is a griefing vector. The attacker's own shares are equally diluted. They always lose at least as much as the victims. Stranded tokens sit in the wrapper, recoverable via a UUPS upgrade.

Proof of Issue

File name: src/flare/implementation/TimelockedVaultWraper.sol
Line number: 56

```
contract TimelockedVaultWrapper is ERC4626Upgradeable,
AccessControlUpgradeable, ReentrancyGuardUpgradeable, UUPSUpgradeable,
PausableUpgradeable {
```

Severity and Impact Summary

The attacker cannot profit and always loses more capital than victims. Requires burning real funds with no economic incentive beyond sabotage. Stranded tokens are recoverable via UUPS upgrade.

Recommendation

Override `mint`, `withdraw`, and `redeem` to revert.

THE EMERGENCYINSTANTREDEEM IS UNREACHABLE DUE TO CONFLICTING MODIFIERS

Finding ID: FYEO-AUOX-06
Severity: **Medium**
Status: **Remediated**

Description

The `activateEmergencyUnwind()` function calls `_pause()`, but `emergencyInstantRedeem()` requires `whenNotPaused`. Since the only way to set `emergencyUnwindActive = true` also pauses the contract, the emergency redemption function can never be called, it is permanently unreachable once the emergency is activated.

Proof of Issue

File name: TimelockedVaultWrapper.sol
Line number: 127

```
function activateEmergencyUnwind() external onlyRole(CONTRACT_OWNER)
whenNotPaused {
    WrapperStorage storage $ = WrapperStorageLib.get();
    require(!$ .emergencyUnwindActive, "Already active");
```

Line number: 268

```
function emergencyInstantRedeem(address receiver, address owner) public
nonReentrant whenNotPaused returns (uint256 assets) {
    WrapperStorage storage $ = WrapperStorageLib.get();
    require($ .emergencyUnwindActive, "Emergency not active");
```

Severity and Impact Summary

The emergency redemption mechanism is unreachable. In an emergency scenario, users cannot redeem their funds through the intended emergency path.

Recommendation

Remove the `whenNotPaused` modifier from `emergencyInstantRedeem()`, or do not pause in `activateEmergencyUnwind()`.

SetManagementFee does not accrue fees before changing the rate

Finding ID: FYEO-AUOX-07
Severity: **Low**
Status: **Remediated**

Description

When the management fee rate is changed via `setManagementFee()`, the function does not first call `_accrueManagementFee()`. This means the new rate is retroactively applied to the entire period since the last accrual. If the fee is increased, the fee recipient receives an unfair windfall. If decreased, they lose legitimately earned fees.

Proof of Issue

File name: TimelockedVaultWrapper.sol
Line number: 345

```
function setManagementFee(uint256 newFeeBPS) external onlyRole(CONTRACT_OWNER)
{
    require(newFeeBPS <= 500, "Max 5%"); // reasonable cap
    WrapperStorage storage $ = WrapperStorageLib.get();
```

Severity and Impact Summary

Incorrect fee accounting on rate changes. The fee recipient either over-collects or under-collects depending on the direction of the rate change.

Recommendation

Call `_accrueManagementFee()` at the start of `setManagementFee()` before updating the rate.

Commit Hash Reference:

For transparency and reference, the security review was conducted on the specific commit hash for the Aureus Ox repository. The commit hash for the reviewed versions is as follows:

a91d705de029c48194eafbcd9d0bead651242c83

Remediations have been submitted with the commit hash b4e8c7e7b0060f6a7afc19b444e1345585848db2.

Conclusion:

In conclusion, the security aspects of the Aureus Ox program remain robust and unaffected by the recent updates. Users can confidently interact with the protocol, assured that their funds are well-protected. The commitment to security exhibited by the development team is commendable, and we appreciate the ongoing efforts to prioritize the safeguarding of user assets.